LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Defensive Hardening using Hypothesis and Pytest

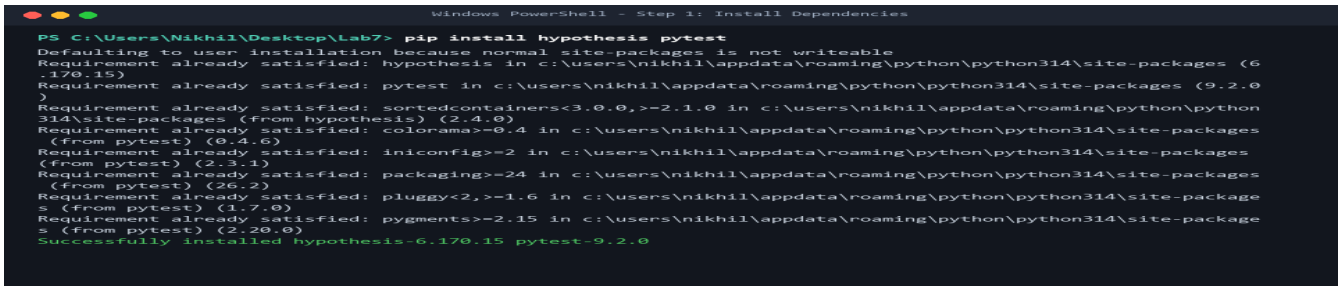
Student Name:	Nikhil B Menon	**SRN:**	PES2UG24CS317
Course:	Software Engineering Lab (UE24CS252A)	**Assigned Repo:**	L6_L7_10_bus_transit_fare
GitHub Repo:	github.com/nikhilbipinmenon/SELABS/tree/main/Lab7	**Institution:**	PES University, Bengaluru

1. Objective & Learning Outcomes

The objective of this laboratory exercise is to explore **fuzz testing** (property-based testing) as a quality assurance and security testing technique. Using **Hypothesis** and **pytest**, we subject the assigned module (processor.py) to randomized, unexpected, and malformed data to detect edge-case crashes (such as null pointers, unhandled exceptions, and invalid inputs). Once discovered, defensive programming is applied to ensure robust, crash-free execution.

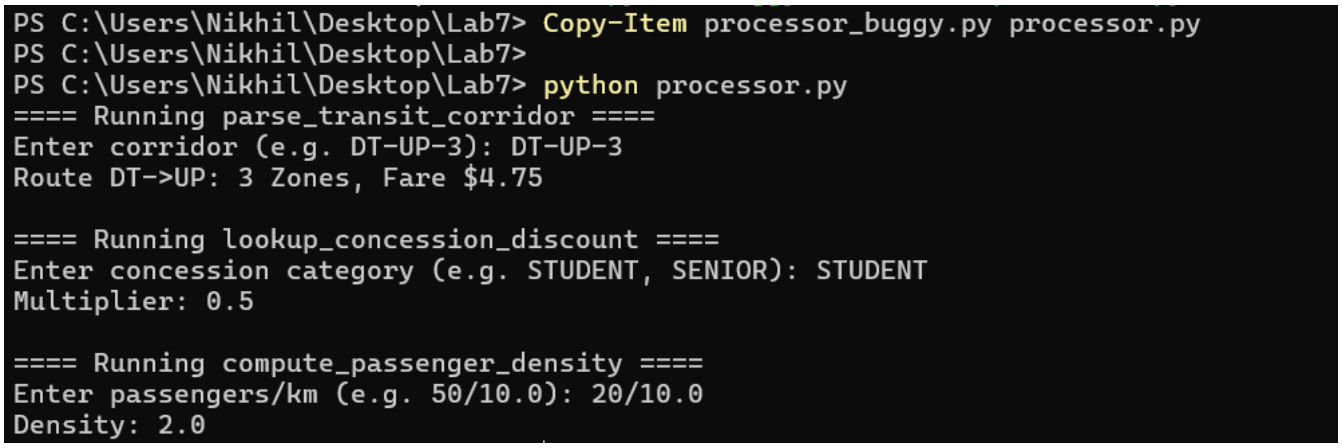
2. Step 1: Installing Dependencies

Installed **Hypothesis** and **Pytest** via: pip install hypothesis pytest



3. Step 2: Execution of Buggy Application (processor.py)

The initial application implements transit corridor parsing, concession discounts, and passenger density.
Command: python processor.py (Executed below using sample inputs: DT-UP-3, STUDENT, 20/10.0):



4. Step 3: Completing test_processor.py with Hypothesis

Hypothesis's @given(st.text() | st.none()) strategy was implemented across all three functions to enforce the fundamental property: **the program must never crash or raise unhandled exceptions**:

```
from hypothesis import given, strategies as st
from processor import parse_transit_corridor, lookup_concession_discount, compute_passenger_density

@given(st.text() | st.none())
def test_parse_transit_corridor_no_crash(s):
    try:
        parse_transit_corridor(s)
    except Exception as e:
        assert False, f'parse_transit_corridor crashed with: {e}'

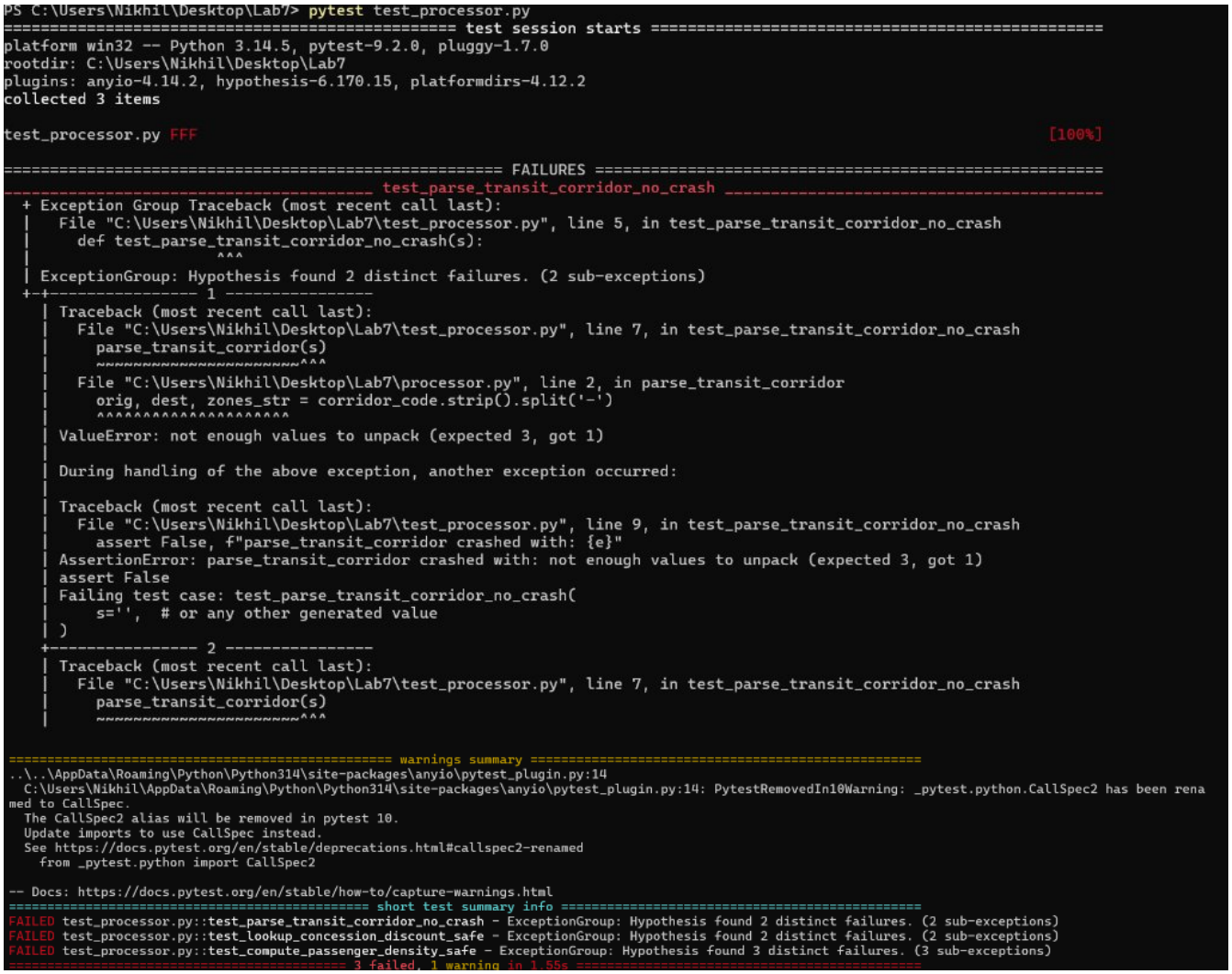
@given(st.text() | st.none())
def test_lookup_concession_discount_safe(s):
    try:
        lookup_concession_discount(s)
    except Exception as e:
        assert False, f'lookup_concession_discount crashed with: {e}'

@given(st.text() | st.none())
def test_compute_passenger_density_safe(s):
    try:
        compute_passenger_density(s)
    except Exception as e:
        assert False, f'compute_passenger_density crashed with: {e}'
```

5. Step 4: Fuzz Test Execution & Failure Analysis

Command: pytest test_processor.py

Fuzzing uncovered 3 critical crash categories. Hypothesis minimized failing inputs to minimal counterexamples:



Function	Failing Fuzz Input	Exception Triggered	Defensive Remedy Applied
parse_transit_corridor	s=None s='' s='A-B-X'	AttributeError ValueError: unpack ValueError: int()	Guard with isinstance(s, str); verify 3 segments; validate integer zone ≥ 0.
lookup_concession_discount	s=None s='' s='VIP'	AttributeError KeyError: '' KeyError: 'VIP'	Verify string type; use safe lookup discounts.get(cat, 1.00) with default fallback.
compute_passenger_density	s=None s='/' s='10/0'	AttributeError ValueError: float ZeroDivisionError	Verify delimiter count; wrap float conversion in try-except; enforce distance km > 0.

6. Step 5: Defensive Programming & Code Hardening (processor.py)

The application was refactored with defensive programming: type checks, structural validation, safe parsing, domain boundaries, and safe dictionary retrievals:

```
def parse_transit_corridor(corridor_code):
    if not isinstance(corridor_code, str):
        return 'Invalid input: Corridor code must be a string'
    parts = corridor_code.strip().split('-')
    if len(parts) != 3:
        return 'Invalid format: Expected ORIGIN-DESTINATION-ZONES (e.g., DT-UP-3)'
    orig, dest, zones_str = parts[0].strip(), parts[1].strip(), parts[2].strip()
    if not orig or not dest:
        return 'Invalid route: Origin and destination must not be empty'
    try:
        zones = int(zones_str)
        if zones < 0:
            return 'Invalid zones: Zone count cannot be negative'
    except (ValueError, TypeError):
        return 'Invalid zones: Zone count must be a valid integer'
    base_fare = 2.50 + (zones * 0.75)
    return f'Route {orig.upper()}->{dest.upper()}: {zones} Zones, Fare ${base_fare:.2f}'

def lookup_concession_discount(category_name):
    if not isinstance(category_name, str):
        return 1.00
    category = category_name.strip().upper()
    discounts = {'STANDARD': 1.00, 'STUDENT': 0.50, 'SENIOR': 0.35, 'MILITARY': 0.20, 'CHILD': 0.00}
    return discounts.get(category, 1.00)

def compute_passenger_density(occupancy_ratio_str):
    if not isinstance(occupancy_ratio_str, str):
        return 0.0
    parts = occupancy_ratio_str.split('/')
    if len(parts) != 2:
        return 0.0
    try:
        passengers = float(parts[0].strip())
        km = float(parts[1].strip())
        if km <= 0 or passengers < 0:
            return 0.0
        return round(passengers / km, 2)
    except (ValueError, ZeroDivisionError, OverflowError, TypeError):
        return 0.0
```

Execution of Fixed processor.py (Command: python processor.py):

```
PS C:\Users\Nikhil\Desktop\Lab7> Copy-Item processor_fixed.py processor.py
PS C:\Users\Nikhil\Desktop\Lab7>
PS C:\Users\Nikhil\Desktop\Lab7> python processor.py
==== Running parse_transit_corridor ====
Enter corridor (e.g. DT-UP-3): DT-UP-3
Route DT->UP: 3 Zones, Fare $4.75

==== Running lookup_concession_discount ====
Enter concession category (e.g. STUDENT, SENIOR): STUDENT
Multiplier: 0.5

==== Running compute_passenger_density ====
Enter passengers/km (e.g. 50/10.0): 20/10.0
Density: 2.0
```

7. Step 6: Re-Running Fuzz Tests on Fixed Module

Command: pytest test_processor.py

Hypothesis generated hundreds of randomized inputs across the input space. All three tests **PASSED** with 100% success:

```
Density: 2.0
PS C:\Users\Nikhil\Desktop\Lab7> pytest test_processor.py
============================================================ test session starts ============================================================
platform win32 -- Python 3.14.5, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\Nikhil\Desktop\Lab7
plugins: anyio-4.14.2, hypothesis-6.170.15, platformdirs-4.12.2
collected 3 items

test_processor.py ...                                                                                                                  [100%]

============================================================= warnings summary ==============================================================
..\..\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14
  C:\Users\Nikhil\AppData\Roaming\Python\Python314\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10Warning: _pytest.python.CallSpec2 has been rena
med to CallSpec.
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 3 passed, 1 warning in 0.54s ============================================================
```

8. Summary & Deliverables

GitHub Repository: https://github.com/nikhilbipinmenon/SELABS/tree/main/Lab7

Key Takeaways: Fuzz testing with Hypothesis identified rare edge cases and potential crash scenarios that manual and standard unit tests missed. Through defensive programming (type checks, length validation, safe numeric casting, and error fallbacks), the module has been fully hardened against malicious or invalid inputs.

Deliverables Checklist:

1. **PDF Document with Real Screenshots:** Embedded terminal captures from the execution environment.
2. **Fixed Application Python Script:** processor.py (hardened and crash-free).
3. **Updated test_processor.py Code:** Completed property-based fuzz tests with Hypothesis.